Informatik und Mathematik vom Fundament aus verstehen

Vom sichtbaren Notizblock zum Algorithmus und zum Programm

Arbeitsstand

Dieses Dokument entwickelt einen didaktischen roten Faden und befindet sich noch im Aufbau. Beispiele und Übungen machen Denkwege sichtbar; weitere Visualisierungen werden schrittweise ergänzt.

Inhaltsverzeichnis

1 Motto **4**

2 Natürliche Sprache präzisieren für Informatik **4**

3 Notizblock – der Speicher **4**

4 Datentypen und Hilfsfunktionen – Was steht in einer Zelle? **6**

4.1 Einfache Datentypen (ein Zeichen pro Zelle) . . . 6

4.2 Zusammengesetzte Typen (mehrere Zellen) . . . 6

5 Tätigkeitenblatt – Abläufe als Bausteine **8**

6 Prozessor – der Ausführer **9**

6.1 1. Einfache Schreibvorgänge . . . 9

6.2 2. Einfache Lesevorgänge . . . 10

6.3 3. Ein einfacher Durchlauf . . . 10

6.4 4. Eine Schleife mit Abfrage . . . 10

7 Problemlösen: vom Ausgangszustand zum Zielzustand **11**

7.1 Problem und Probleminstanz . . . 12

7.2 Ausgangszustand (Startsituation) und gewünschter Zielzustand . . . 12

7.3 Sinnvolle Werkzeuganwendungen im aktuellen Zustand . . . 12

7.4 Ein Lösungsweg mithilfe der Werkzeuge . . . 12

8 Algorithmus und Programm **13**

8.1 Eigenschaften eines Algorithmus . . . 13

8.2 Unterschied zwischen Algorithmus und Programm . . . 14

9 Rechnen mit integers – ein erster Algorithmus 14

9.1 Grundidee . . . 14
9.2 Beispiel: Addition mit Übertrag . . . 15
9.3 Beispiel: Subtraktion mit Übertrag / Ausleihen . . . 16
9.4 Warum ist das ein Notizblock-Maschinen-Schritt? . . . 16
9.5 Erkenntnis . . . 16

10 Hilfsblock – Hilfsvariablen 16

11 Programmatische Problemlösung mithilfe von Werkzeugen 19

11.1 Demonstrationsbeispiel . . . 19
11.1.1 Problem und Probleminstanz . . . 19
11.1.2 Ausgangszustand (Startsituation) . . . 19
11.1.3 Gewünschter Zielzustand . . . 19
11.1.4 Werkzeuge und begründete Werkzeuganwendungen . . . 20
11.1.5 Erreichter Zielzustand (Endsituation) . . . 20

12 Laufzeitanalyse: vom Schrittmodell zur Größenordnung 21

12.1 Das Schrittmodell: Was kostet wie viel? . . . 21
12.2 Konkrete Schrittzahl der linearen Suche . . . 21
12.3 Konkrete Schrittzahl der Binärsuche . . . 21
12.4 Von der Schrittzahl zur asymptotischen Laufzeitordnung . . . 22
12.4.1 Die Idee der Asymptote: Kleingeld spielt keine Rolle . . . 22
12.5 Wachstumsordnungen vergleichen . . . 23
12.6 Verbindung zur JavaScript-Umsetzung . . . 23

13 Sanfter Umstieg in eine Hochsprache (JavaScript) 24

13.1 Stufe 1: Gleicher Speicher, andere Schreibweise . . . 24
13.2 Stufe 2: Lesen und Schreiben . . . 24
13.3 Stufe 3: Mehrere Zeichen zu einem Wert zusammenfassen . . . 25
13.4 Stufe 4: Wiederholen und Entscheiden . . . 25
13.5 Stufe 5: Das Tätigkeitenblatt wird zur Funktion . . . 26

14 Zusammenhang Informatik - Mathematik **27**
14.1 Gleichungen über Zahlen . . . 27
14.1.1 Warum Umkehrfunktionen häufig sinnvoller sind . . . 27
14.2 Gleichungen über nicht-numerische Objekte: Farben . . . 28
14.3 Gleichungssysteme über bewegende Objekte . . . 29
14.4 Schlussfolgerungen . . . 30
14.4.1 Was ist eine Schlussfolgerung? . . . 30
14.4.2 Beispiele aus der natürlichen Sprache . . . 30
14.5 Beweisführung . . . 30
14.5.1 Was ist eine Beweisführung? . . . 30
14.5.2 Beispiel 1 – Sokrates . . . 31
14.5.3 Beispiel 2 – Arithmetik: Quadratische Faktorisierung . . . 31
14.5.4 Erkenntnis . . . 32
14.5.5 Beispiel 3 - Teilbarkeit prüfen mit Polynom-Kongruenz . . . 32
14.5.6 Beispiel 4 - Prüfung der Geradzahligkeit . . . 34

15 Wie kann man das Kochen lernen? **34**

16 Tipps zur Didaktik **34**
16.1 Wo Übungen in diesem Lernweg sinnvoll sind . . . 34

1 Motto

Unser Gehirn kann die gleichen grundlegenden Operationen - Daten schreiben/lesen, Schleifen, Abfragen - wie ein Prozessor ausführen.
Jeder noch so komplizierte Algorithmus (wozu auch kreative Problemlösungen zählen) könnte alleine mit diesen Grundoperationen umgesetzt werden.

2 Natürliche Sprache präzisieren für Informatik

Wir Menschen nutzen unsere **natürliche Sprache** – zum Beispiel Deutsch oder Englisch –, um Sachverhalte zu beschreiben.
Doch viele Sätze (und sogar einzelne Begriffe) können unterschiedlich verstanden werden.
Für die Informatik und Mathematik ist das ein Problem, denn dort müssen Beschreibungen eindeutig sein.
Deshalb verwendet man Programmiersprachen. Und bevor wir uns großen Sprachen widmen,
beginnen wir mit einer sehr einfachen, klar definierten Sprache, mit der sich Inhalte präzise und ohne Mehrdeutigkeit beschreiben lassen.

3 Notizblock – der Speicher

Der Notizblock ist unser **Speicher**: Er enthält alle Informationen, die wir geordnet ablegen und wieder auslesen können.

Wir nutzen **kariertes Papier**, auf dem wir Informationen an beliebigen Positionen notieren können.
Jede Zelle hat eine Adresse `[Zeile, Spalte]` – genau wie beim handschriftlichen Schreiben auf kariertem Papier.
Grundsätzlich kann man die Informationen genauso wie auf einem herkömmlichen Notizblock anordnen.

Beispiel für eine mögliche tabellarische Organisation:

```
# Kariertes Papier - Beispiel einer Tabellenstruktur
# (Man könnte die Informationen aber auch ganz anders anordnen)
# Jede Zeile steht für einen Eintrag, jede Spalte für ein Informationsfeld
# Wichtig: Wie beim karierten Zettel kommt jedes Zeichen (Buchstabe, Ziffer, Symbol) in eine eigene Zelle.
# Beispiel: Einkaufszettel (Mengen, Einheiten und Ladennamen zeichenweise pro Zelle)

         | Sp.1 | Sp.2 | Sp.3 | Sp.4 | Sp.5 | Sp.6   | Sp.7 | Sp.8 | Sp.9 | Sp.10| Sp.11
---------+------+------+------+------+------+--------+------+------+------+------+------
Zeile 1  | [1,1]| [1,2]| [1,3]| [1,4]| [1,5]| [1,6]  | [1,7]| [1,8]| [1,9]|[1,10]|[1,11]
(Mehl)   | MG1  | MG2  | EH1  | EH2  | EH3  | (leer) | L1   | L2   | L3   | L4   | L5
         | 1    | 2    | k    | g    |      |        | E    | d    | e    | k    | a

Zeile 2  | [2,1]| [2,2]| [2,3]| [2,4]| [2,5]| [2,6]  | [2,7]| [2,8]| [2,9]|[2,10]|[2,11]
(Zucker) | MG1  | MG2  | EH1  | EH2  | EH3  | (leer) | L1   | L2   | L3   | L4   | L5
         | 0    | 5    | k    | g    |      |        | R    | e    | w    | e    |

Zeile 3  | [3,1]| [3,2]| [3,3]| [3,4]| [3,5]| [3,6]  | [3,7]| [3,8]| [3,9]|[3,10]|[3,11]
(Eier)   | MG1  | MG2  | EH1  | EH2  | EH3  | (leer) | L1   | L2   | L3   | L4   | L5
         | 1    | 0    | S    | t    | k    |        | A    | l    | d    | i    |
```

```
Einkaufszettel (Beispiel):
Menge aus zwei Ziffernspalten:  [MG1|MG2] = [1|2] -> 12
Einheit aus Buchstabenspalten:  [EH1|EH2] = [k|g] -> "kg", [EH1|EH2|EH3] = [S|t|k] -> "Stk"
Sp.6 bleibt leer (visuelle Trennung zwischen Einheit und Laden)
Laden aus Buchstabenspalten:    [L1|...|L5] = [E|d|e|k|a] -> "Edeka"
Hinweis: Leere Zellen (wie EH3 bei Mehl/Zucker, L5 bei Rewe/Aldi) bleiben einfach frei.

# Beispiel: Bibliotheksliste (alle Zeichen in einzelnen Zellen)

Zeile 1  | [1,1]| [1,2]| [1,3]| [1,4]| [1,5]| [1,6]| [1,7]| [1,8] | [1,9]|[1,10]|[1,11]|[1,12]|[1,13]|[1,14] |[1,15]|[1,16]|[1,17]|[1,18]|[1,19]
(Buch)   | T1   | T2   | T3   | T4   | T5   | T6   | T7   |       | Nr1  | Nr2  | Nr3  | Nr4  | Nr5  |       | PV1  | PV2  | PK   | PN1  | PN2
         | M    | a    | t    | h    | e    |      | 7    |       | 9    | 7    | 8    | 3    | 1    |       |      | 1    | ,    | 9    | 9

Zeile 2  | [2,1]| [2,2]| [2,3]| [2,4]| [2,5]| [2,6]| [2,7]| [2,8] | [2,9]|[2,10]|[2,11]|[2,12]|[2,13]|[2,14] |[2,15]|[2,16]|[2,17]|[2,18]|[2,19]
(Buch)   | T1   | T2   | T3   | T4   | T5   | T6   | T7   |       | Nr1  | Nr2  | Nr3  | Nr4  | Nr5  |       | PV1  | PV2  | PK   | PN1  | PN2
         | P    | h    | y    | s    | i    | k    |      |       | 9    | 3    | 4    | 1    | 0    |       | 2    | 2    | ,    | 4    | 9

Bibliotheksliste (Beispiel):
Titel aus Buchstabenspalten: [T1|...|T7] = [M|a|t|h|e| |7] -> "Mathe 7" (Leerzeichen ist auch ein Zeichen!)
Zwischen Titel, Nummer und Preis bleibt jeweils eine Zelle als optischer Abstand leer.
Nummer aus Ziffernspalten:   [Nr1|...|Nr5] = [9|7|8|3|1] -> 97831
Preisfeld mit fünf Zellen:    [PV1|PV2|PK|PN1|PN2]
1,99  = [ |1|,|9|9] (erste Vorkommastelle bleibt leer)
22,49 = [2|2|,|4|9]

# Beispiel: Kalender (Datum als einzelne Ziffern pro Zelle)

         | Spalte 1  | Spalte 2 | Spalte 3 | Spalte 4 | Spalte 5 | Spalte 6 | Spalte 7 | Spalte 8
---------+-----------+----------+----------+----------+----------+----------+----------+----------
Zeile 1  | [1,1]     | [1,2]    | [1,3]    | [1,4]    | [1,5]    | [1,6]    | [1,7]    | [1,8]
(Termin) | T_D1      | T_D2     | M_D1     | M_D2     | J_D1     | J_D2     | J_D3     | J_D4
         | 2         | 3        | 0        | 5        | 2        | 0        | 2        | 6

Zeile 2  | [2,1]     | [2,2]    | [2,3]    | [2,4]    | [2,5]    | [2,6]    | [2,7]    | [2,8]
(Termin) | T_D1      | T_D2     | M_D1     | M_D2     | J_D1     | J_D2     | J_D3     | J_D4
         | 1         | 1        | 0        | 6        | 2        | 0        | 2        | 6

Kalender (Beispiel):
Tag aus [T_D1|T_D2] = [2|3] -> 23
Monat aus [M_D1|M_D2] = [0|5] -> 05
Jahr aus [J_D1|J_D2|J_D3|J_D4] = [2|0|2|6] -> 2026
Gesamt: 23.05.2026
```

Diese tabellarische Organisation ist nur eine von vielen Möglichkeiten. Entscheidend dabei: **jedes Zeichen** – ob Buchstabe, Ziffer oder Leerzeichen – belegt genau eine Zelle. Das gilt

nicht nur für Ziffern, sondern für alle Inhalte.

Wichtig ist: Jede Zelle hat eine eindeutige Adresse `[Zeile, Spalte]`, sodass Programme die Informationen gezielt lesen und schreiben können.
Der Notizblock funktioniert damit wie der Speicher einer Maschine – **flexibel, übersichtlich und maschinenverständlich**.

Übung: Einen Familienstammbaum gliedern

Überlege dir, wie sich ein Familienstammbaum auf dem karierten Notizblock speichern lässt, **ohne Verbindungslinien zu verwenden**. Lege fest, welche Informationen zu einer Person gehören und wie Eltern, Kinder oder Generationen allein durch Zellen und deren Inhalte zugeordnet werden können. Zeichne eine mögliche Belegung für mindestens drei Generationen und erkläre, wie man zu einer Person ihre Eltern findet. Es gibt mehrere sinnvolle Lösungen.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

4 Datentypen und Hilfsfunktionen – Was steht in einer Zelle?

Bisher haben wir Inhalte einfach in Zellen geschrieben, ohne genauer zu sagen, **was für eine Art von Inhalt** dort steht.
In der Informatik ist das aber wichtig: Jede Zelle enthält einen Wert eines bestimmten **Datentyps**.
Der Datentyp legt fest, welche Werte erlaubt sind und wie mit ihnen gearbeitet werden kann.

4.1 Einfache Datentypen (ein Zeichen pro Zelle)

Die drei grundlegenden Typen für eine einzelne Zelle sind:

- **char** (Zeichen): Eine Zelle enthält genau **einen** Buchstaben, eine Ziffer oder ein Symbol.
  Beispiele: `[1,1] = 'T'`, `[1,2] = 'o'`, `[3,3] = '?'`, `[2,5] = ' '` (Leerzeichen ist auch ein char!)
- **digit** (Ziffer): Eine Zelle enthält genau **eine** Ziffer von 0 bis 9.
  Beispiele: `[1,1] = 1`, `[1,2] = 2`, `[2,1] = 0`
  Hinweis: digit ist ein Sonderfall von char – eine Ziffer ist auch ein Zeichen.
- **boolean** (Wahrheitswert): Eine Zelle enthält genau **einen** von zwei möglichen Werten: `ja` oder `nein` (bzw. `wahr` / `falsch`).
  Beispiele: `h[11,1] = nein` (Merker „gefunden"), `h[12,1] = ja` (Merker „fertig")
  Typische Verwendung: Zustände und Merker im Hilfsblock.

4.2 Zusammengesetzte Typen (mehrere Zellen)

Einzelne chars und digits sind oft unhandlich – wir wollen Wörter, Zahlen oder Kommazahlen speichern.
Dafür führen wir **Hilfsfunktionen** ein, die mehrere Zellen lesen oder befüllen und den Inhalt als Ganzes übergeben.
Sie sind sozusagen **Abkürzungen für wiederkehrende Abläufe** auf dem Notizblock.

Auch bei Anwendung dieser Hilfsfunktionen wird ein String oder Integer **nicht in einer einzigen Zelle zusammengefasst**. Beim Schreiben verteilt die Hilfsfunktion den Wert automatisch Zeichen für Zeichen beziehungsweise Ziffer für Ziffer auf mehrere aufeinanderfolgende Zellen. Beim Lesen setzt sie die Inhalte mehrerer Zellen lediglich für die Übergabe wieder zu einem String oder Integer zusammen. Die Speicherung im Notizblock bleibt also zellenweise; man muss sich nur nicht mehr selbst um jeden einzelnen Lese- oder Schreibschritt kümmern.

- **string(zeile, spalte, zeichenkette)** – Zeichenkette schreiben:
  Verteilt die angegebene Zeichenkette ab der Startzelle Zeichen für Zeichen auf aufeinanderfolgende Zellen.
  string(zeile, spalteVon, spalteBis) – Zeichenkette lesen:
  Liest alle Zellen von *spalteVon* bis *spalteBis* und gibt sie als Zeichenkette zurück.
  Beim Lesen ist die Zeichenkette unbekannt – deshalb gibt man den Bereich explizit an.

```
string schreiben - string(1, 7, "Edeka"):
-> [1,7]='E', [1,8]='d', [1,9]='e', [1,10]='k', [1,11]='a'

string lesen - string(1, 7, 11):
-> liest [1,7] bis [1,11] -> "Edeka"
```

- **integer(zeile, spalte, zahl)** – Ganze Zahl schreiben:
  Verteilt die angegebene Zahl ab der Startzelle Ziffer für Ziffer auf aufeinanderfolgende Zellen.
  integer(zeile, spalteVon, spalteBis) – Ganze Zahl lesen:
  Liest alle Zellen von *spalteVon* bis *spalteBis* und setzt sie zur vollständigen Zahl zusammen.

```
integer schreiben - integer(1, 9, 97831):
-> [1,9]=9, [1,10]=7, [1,11]=8, [1,12]=3, [1,13]=1

integer lesen - integer(1, 9, 13):
-> liest [1,9] bis [1,13] -> 97831
```

- **float(zeile, spalte, zahl)** – Fließkommazahl schreiben:
  Wie integer, aber im Notizblock steht auch das Komma als eigenes Zeichen.
  In manchen Hilfsdarstellungen kann das Komma weggelassen werden; im eigentlichen Notizblock-Modell wird es aber mitgeschrieben.
  float(zeile, spalteVon, spalteBis, kommaPosition) – Fließkommazahl lesen:
  Liest alle Zellen von *spalteVon* bis *spalteBis* und setzt das Komma an der angegebenen Stelle wieder ein.

```
float schreiben - float(1, 16, 1,99):
-> [1,16]=1, [1,17]=',', [1,18]=9, [1,19]=9

float lesen - float(1, 16, 19, 1):
-> liest [1,16] bis [1,19], Komma nach Stelle 1 -> 1,99

float-Darstellung ohne Komma (vereinfachte Skizze):
-> [1,16]=1, [1,17]=9, [1,18]=9
```

Hinweis:
Wir wollen mit den Zahlen auch rechnen können. Dies müssen wir dem Computer auch beibringen.
Vorerst stellen wir uns aber vor, dass wir mithilfe unseres eigenen Verstands mit den Zahlen auch rechnen können.

Erkenntnis:
Der Notizblock kennt selbst keine zusammengesetzten Werte – er enthält nur einzelne chars, digits oder booleans pro Zelle.
Wörter, Zahlen und Kommazahlen entstehen erst durch das **gezielte Lesen und Zusammensetzen mehrerer Zellen** mithilfe dieser Hilfsfunktionen.
Genau das werden wir später auch in JavaScript so umsetzen: Daten zeichenweise im Array ablegen und per Funktion auslesen.

Übung: Dieselbe Information verschieden darstellen

Speichere den Text „Bus 42ünd den Preis „3,20ßunächst Zeichen für Zeichen. Lege danach fest, welche Hilfsfunktionen beide Werte wieder zusammensetzen. Begründe für jede Zelle den gewählten Datentyp.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

5 Tätigkeitenblatt – Abläufe als Bausteine

Neben **Notizblock** (Daten) und **Hilfsblock** (Hilfsvariablen) ist ein **Tätigkeitenblatt** sinnvoll.
Dort stehen keine konkreten Werte, sondern **rezeptartige Abläufe**, die der Prozessor bei Bedarf ausführen kann.

Wozu dient das Tätigkeitenblatt?

- **Wiederverwendung:** Ein Ablauf wird einmal beschrieben und mehrfach genutzt.
- **Übersicht:** Lange Lösungen werden in kleine, verständliche Bausteine zerlegt.
- **Fehlersuche:** Man prüft gezielt einen Baustein statt das ganze Gesamtprogramm.

Möglicher Aufbau eines Eintrags:

- **Name der Tätigkeit** (z. B. *nummer_lesen*)
- **Startwerte/Parameter** (z. B. Zeile = 1)
- **Schritte** (rezeptartig, eindeutig)
- **Ergebnis** (z. B. zusammengesetzte Nummer im Notizblock)

Beispiel auf Basis der Bibliotheksliste:

```
Tätigkeit: nummer_lesen(zeile)

1) Setze [10,1] bis [10,5] jeweils auf "".
2) Lies in der angegebenen Zeile die Spalten 9 bis 13.
3) Schreibe die gelesenen Ziffern der Reihe nach in [10,1] bis [10,5].
4) Ergebnis: Ab [10,1] steht die vollständige Nummer zellenweise.

Tätigkeit: preis_lesen(zeile)

1) Setze [11,1] bis [11,5] jeweils auf "".
```

```
2) Lies in der angegebenen Zeile die Spalten 15 bis 19.
3) Schreibe jedes gelesene Zeichen in die entsprechende Zelle [11,1] bis [11,5].
4) Ergebnis: Ab [11,1] steht der Preis einschließlich Komma zellenweise.

Tätigkeit: artikel_leeren(zeile)

1) Gehe in der angegebenen Zeile jede Zelle von Spalte 1 bis 19 durch.
2) Setze jede dieser Zellen auf "".
3) Ergebnis: Die gesamte Artikelzeile ist leer und kann neu belegt werden.

Tätigkeit: artikel_hinzufügen(zeile, titel, nummer, preis)

1) Führe artikel_leeren(zeile) aus.
2) Schreibe den Titel zeichenweise ab Spalte 1.
3) Schreibe die Nummer ziffernweise in die Spalten 9 bis 13.
4) Schreibe den Preis einschließlich Komma in die Spalten 15 bis 19.
5) Ergebnis: Die angegebene Zeile enthält den neuen Artikel.
```

Didaktisch ist das der nächste logische Schritt: Lernende sehen, dass ein Programm nicht nur aus Daten besteht,
sondern auch aus **wiederverwendbaren Tätigkeiten**. In einer späteren JavaScript-Umsetzung entsprechen diese Tätigkeiten dann typischerweise **Funktionen**.

Übung: Eine Nummer rückwärts lesen

Entwirf die Tätigkeit *reverse_nummer_lesen(zeile)*. Sie soll die Nummer aus den Spalten 9 bis 13 lesen und in umgekehrter Reihenfolge ab `[10,1]` ablegen. Gib an, welche Zielzellen zuerst geleert werden, welche Quellzelle in welche Zielzelle geschrieben wird und welches Ergebnis für die Nummer 97831 entsteht.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

6 Prozessor – der Ausführer

Der **Prozessor** (bspw. unser Gehirn) liest den Notizblock, führt Befehle aus und schreibt Ergebnisse zurück.
Er kennt vier grundlegende Fähigkeiten: **Schreiben**, **Lesen**, **Falls-Abfrage** und **Schleife**.

6.1 1. Einfache Schreibvorgänge

Am einfachsten beginnt man mit reinen Schreibvorgängen.
Der Prozessor trägt also Werte an bestimmten Stellen in den Notizblock ein.

Beispiel auf Basis des Einkaufszettels:

- Trage für Mehl in Zeile 1, Spalte 1 die erste Ziffer der Menge ein: 1.
- Trage für Mehl in Zeile 1, Spalte 2 die zweite Ziffer der Menge ein: 2.

- Trage für Zucker in Zeile 2, Spalte 1 die erste Ziffer der Menge ein: 0.
- Trage für Zucker in Zeile 2, Spalte 2 die zweite Ziffer der Menge ein: 5.

Hier geht es nur darum zu verstehen: **Der Prozessor kann gezielt in einzelne Zellen schreiben.**

6.2 2. Einfache Lesevorgänge

Danach folgt das Lesen. Der Prozessor schaut an einer bestimmten Stelle nach, welcher Wert dort steht.

Beispiel auf Basis des Einkaufszettels:

- Lies bei Zucker die erste Mengenziffer aus Zeile 2, Spalte 1.
- Lies bei Zucker die zweite Mengenziffer aus Zeile 2, Spalte 2.
- Lies bei Eiern die Einheit aus Zeile 3, Spalte 3 bis 5: `string(3, 3, 5)`.
- Lies bei Mehl den Laden aus Zeile 1, Spalte 7 bis 11: `string(1, 7, 11)`.

Auch hier ist die Idee noch einfach: **Der Prozessor kann gezielt aus einzelnen Zellen lesen.**

6.3 3. Ein einfacher Durchlauf

Nun kann der Prozessor mehrere Einträge der Reihe nach abarbeiten.
Das ist bereits eine **Schleife**: Derselbe Ablauf wird für mehrere Zeilen und Zellen wiederholt.

Beispiel auf Basis der Bibliotheksliste:

Wir gehen jede einzelne Zeile und pro Zeile jede einzelne Zelle durch:

```
Beispiel eines einfachen Durchlaufs in natürlicher Sprache:

Für jede Zeile (= jedes Buch):
  Für jede Zelle der Zeile (= Titel, dann jede Ziffer der Nummer, dann jede Ziffer des Preises):
    Lies den Wert der aktuellen Zelle.
    Wenn wir uns in Sp. 9 bis 13 befinden, sammeln wir die Ziffern der Nummer.
    Wenn wir uns in Sp. 15 bis 19 befinden, sammeln wir die Zeichen des Preises.
```

So wird klar: **Der Prozessor kann mehrere Zeilen nacheinander bearbeiten – und innerhalb jeder Zeile Zelle für Zelle vorgehen.**

6.4 4. Eine Schleife mit Abfrage

Nun erweitern wir die Schleife um eine **Abfrage**.
Dabei wird beim wiederholten Durchlaufen zusätzlich geprüft, ob ein bestimmter Fall eingetreten ist.

Beispiel auf Basis der Bibliotheksliste:

Wir gehen wieder jede einzelne Zeile und pro Zeile jede einzelne Zelle durch:

```
Beispiel einer Schleife mit Abfrage in natürlicher Sprache:

Für jede Zeile (= jedes Buch):
  Für jede Zelle der Zeile (= Titel, dann jede Ziffer der Nummer, dann jede Ziffer des Preises):
    Lies den Wert der aktuellen Zelle.
    Wenn wir uns in Sp. 9 bis 13 befinden, sammeln wir die Ziffern der Nummer.
    Wenn wir uns in Sp. 15 bis 19 befinden, sammeln wir die Zeichen des Preises.
  Prüfe nach dem Einlesen der Zeile, ob die gelesene Nummer der gesuchten Nummer 97831 entspricht.
  Falls ja, schreibe die Nummer und den Preis in die dafür vorgesehene Zeile 10.
  (Auch dort müssen die Ziffern wieder spaltenweise in die einzelnen Zellen eingetragen werden.)
  Falls nein, gehe zur nächsten Zeile weiter.
```

Hier kommen mehrere Fähigkeiten zusammen: **lesen**, **prüfen**, **gegebenenfalls schreiben** und **wiederholen** – und zwar auf zwei Ebenen: einmal über die Zeilen, einmal über die Zellen pro Zeile.

Verschachtelung bedeutet, dass ein Ablauf innerhalb eines anderen Ablaufs liegt. Hier wird für jede Zeile eine innere Schleife über deren Zellen ausgeführt. Der Prozessor beginnt mit der ersten Zeile, arbeitet die innere Zellschleife vollständig ab und setzt erst danach die äußere Schleife mit der nächsten Zeile fort. Trifft er innerhalb der inneren Schleife auf eine Bedingung, wertet er diese an der aktuellen Zelle aus und kehrt anschließend an die passende Stelle der inneren Schleife zurück.

So wird zunächst in natürlicher Sprache deutlich, **was** der Prozessor tun soll.
Erst im nächsten Schritt wird daraus eine formale Programmiersprache.
Dadurch bleibt der Einstieg anschaulich und verständlich.

Übung: Den nächsten Schritt begründen

Beginne eine Suche nach einem frei gewählten Buchtitel. Halte nach jeder Werkzeuganwendung den aktuellen Zustand fest und entscheide zwischen Lesen, Schreiben, Bedingung und Schleife. Begründe jeweils, warum das gewählte Werkzeug jetzt sinnvoll ist.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

Übung: Zusammengesetzte Typen verwenden

Formuliere das Beispiel aus diesem Abschnitt neu, indem du die Hilfsfunktionen für zusammengesetzte Typen verwendest. Kennzeichne, welche Schleife dadurch entfällt.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

Die Didaktik soll darauf beruhen, dass die Schulmathematik mithilfe eines einfachen JavaScript-Modells mit Matrix als Notizblock beschrieben werden kann.

7 Problemlösen: vom Ausgangszustand zum Zielzustand

Eine **Aufgabe** gibt vor, was getan werden soll. Sie kann auch dann bearbeitet werden, wenn der Lösungsweg bereits bekannt ist.
Ein **Problem** liegt dagegen vor, wenn ein gewünschtes Ziel bekannt ist, aber noch kein unmittelbar anwendbarer Lösungsweg zur Verfügung steht.
Problemlösen bedeutet deshalb, einen geeigneten Weg vom vorhandenen Zustand zum gewünschten Zustand zu finden.

7.1 Problem und Probleminstanz

Ein Problem beschreibt eine ganze Klasse gleichartiger Fragestellungen.
Eine **Probleminstanz** ist ein einzelner, konkreter Fall dieses Problems.

- **Problem:** Finde eine bestimmte Nummer in einer Bibliotheksliste.
- **Probleminstanz:** Finde die Nummer `97831` in der aktuell vorliegenden Liste.

Die Unterscheidung ist wichtig: Ein Lösungsweg soll nach Möglichkeit nicht nur eine einzige Probleminstanz lösen, sondern für alle zulässigen Instanzen des Problems geeignet sein.

7.2 Ausgangszustand (Startsituation) und gewünschter Zielzustand

Der **Ausgangszustand**, in Beispielen auch **Startsituation** genannt, beschreibt alle zu Beginn bekannten und relevanten Informationen. Dazu gehören die Eingaben, ihre Anordnung im Notizblock und mögliche Voraussetzungen.
Der **gewünschte Zielzustand** beschreibt, woran eine erfolgreiche Lösung erkannt wird. Er legt fest, welches Ergebnis am Ende vorliegen und wo es abgelegt werden soll. Ist dieser Zustand erreicht, nennen wir ihn in Beispielen auch **Endsituation**.

- **Ausgangszustand:** Im Notizblock steht eine Bibliotheksliste; die gesuchte Nummer lautet `97831`.
- **Zielzustand bei einem Treffer:** Der passende Eintrag wurde gefunden und seine Position oder sein Preis steht an einer festgelegten Stelle.
- **Zielzustand ohne Treffer:** Der gesamte relevante Bereich wurde geprüft und der Merker *gefunden* steht auf *nein*.

Ein präziser Zielzustand verhindert, dass ein Ablauf zu früh beendet wird oder unklar bleibt, ob das Problem tatsächlich gelöst wurde.

7.3 Sinnvolle Werkzeuganwendungen im aktuellen Zustand

Zum Lösen verwenden wir die bereits bekannten Werkzeuge **Lesen**, **Schreiben**, **Bedingung** und **Schleife**.
Die konkrete Verwendung eines Werkzeugs nennen wir eine **Werkzeuganwendung**. Eine Werkzeuganwendung verändert entweder den aktuellen Zustand oder liefert Informationen, die die nächste Werkzeuganwendung bestimmen.

Nicht jede grundsätzlich erlaubte Werkzeuganwendung ist in jedem Zustand sinnvoll. Vor jeder Anwendung können wir daher fragen:

- **Ist die Anwendung ausführbar?** Die benötigte Zelle oder Information muss vorhanden sein.
- **Bringt die Anwendung uns dem Ziel näher?** Sie sollte neue relevante Informationen liefern oder den Zustand gezielt verändern.
- **Bewahrt die Anwendung wichtige Bedingungen?** Bereits korrekt gespeicherte Daten dürfen nicht unbeabsichtigt zerstört werden.
- **Passt die Anwendung zum aktuellen Zustand?** Nach einem Treffer ist beispielsweise ein weiterer Suchschritt meist unnötig.

Bei der Suche wäre es etwa nicht sinnvoll, sofort *gefunden = ja* zu schreiben. Zuerst muss ein Eintrag gelesen und mit der gesuchten Nummer verglichen werden. Erst wenn die Bedingung erfüllt ist, ist diese Werkzeuganwendung sinnvoll.

7.4 Ein Lösungsweg mithilfe der Werkzeuge

Ein Lösungsweg ist eine geordnete Folge sinnvoller Werkzeuganwendungen, die den Ausgangszustand in den Zielzustand überführt. Für die Suche könnte er zunächst in natürlicher Sprache so aussehen:

1. Schreibe *gefunden* = *nein* in den Hilfsblock und beginne beim ersten Eintrag.
2. Lies die Nummer des aktuellen Eintrags.
3. Vergleiche sie mit der gesuchten Nummer.
4. Falls beide Nummern gleich sind, schreibe *gefunden* = *ja* und merke die Position.
5. Falls sie nicht gleich sind und noch ein Eintrag folgt, gehe zum nächsten Eintrag und wiederhole die Schritte 2 bis 4.
6. Falls kein Eintrag mehr folgt, beende die Suche mit *gefunden* = *nein*.

Problemlösen besteht damit nicht nur darin, Werkzeuge zu kennen. Entscheidend ist, in jedem Zustand eine passende nächste Werkzeuganwendung auszuwählen und die Anwendungen so anzuordnen, dass der Zielzustand tatsächlich erreicht wird.

Übung: Einen Streit mit der Freundin auflösen

Betrachte einen Streit mit deiner Freundin als konkrete Probleminstanz. Beschreibe den **Ausgangszustand**, ohne bereits eine Lösung vorwegzunehmen. Welche Werkzeuge stehen zur Verfügung, beispielsweise Zuhören, Nachfragen, die eigene Sicht erklären, eine Pause vereinbaren oder sich entschuldigen? Formuliere den **gewünschten Zielzustand**. Begründe anschließend einen möglichen Lösungsweg und prüfe bei jedem Werkzeugeinsatz, ob er im aktuellen Zustand sinnvoll ist. Es kann mehrere geeignete Lösungswege geben.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

Übung: Eine Aufgabe als Problem strukturieren

Wähle eine Alltagssituation, etwa das Finden des günstigsten Produkts. Formuliere Problem, eine konkrete Probleminstanz, Ausgangszustand und Zielzustand. Nenne verfügbare Werkzeuge und skizziere einen eigenen Lösungsweg. Mehrere korrekte Wege sind ausdrücklich möglich.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

8 Algorithmus und Programm

Ist ein Lösungsweg so genau beschrieben, dass seine Schritte eindeutig ausgeführt werden können, sprechen wir von einem **Algorithmus**.
Ein Algorithmus ist eine allgemeine, eindeutige und endliche Handlungsvorschrift zur Lösung eines Problems.

8.1 Eigenschaften eines Algorithmus

Ein Algorithmus besitzt mindestens folgende Eigenschaften:

- **Allgemeinheit:** Er löst nicht nur ein einzelnes Beispiel, sondern alle vorgesehenen Probleminstanzen.
- **Eindeutigkeit:** In jedem Schritt ist festgelegt, was als Nächstes zu tun ist.
- **Ausführbarkeit:** Jeder Einzelschritt kann mit den verfügbaren Werkzeugen ausgeführt werden.
- **Endlichkeit:** Die Beschreibung besteht aus endlich vielen Anweisungen.

- **Terminierung:** Für jede zulässige Eingabe endet die Ausführung nach endlich vielen Schritten.
- **Korrektheit:** Wenn der Algorithmus endet, entspricht sein Ergebnis dem festgelegten Zielzustand.

Der Algorithmus beschreibt dabei die Idee und die Regeln des Lösungswegs. Er ist noch nicht an eine bestimmte Programmiersprache oder einen bestimmten Computer gebunden. Er kann beispielsweise in präziser natürlicher Sprache, als Ablaufdiagramm oder als Pseudocode dargestellt werden.

8.2 Unterschied zwischen Algorithmus und Programm

Ein **Programm** ist die konkrete Formulierung eines Algorithmus in einer Programmiersprache. Es berücksichtigt die Syntax dieser Sprache sowie die dort vorhandenen Datentypen, Speicherstrukturen und Befehle.
Mehrere unterschiedliche Programme können daher denselben Algorithmus umsetzen, etwa eines in JavaScript und ein anderes in Pascal.

Begriff	**Algorithmus**	**Programm**
Zweck	Allgemeiner Lösungsweg für ein Problem	Konkrete, vom Computer ausführbare Umsetzung
Darstellung	Natürliche Sprache, Pseudocode oder Ablaufdiagramm	Programmiersprache, zum Beispiel JavaScript
Abhängigkeit	Grundsätzlich unabhängig von einer Programmiersprache	An Sprache und Ausführungsumgebung gebunden
Beispiel	Prüfe Listeneinträge nacheinander auf Übereinstimmung	Eine JavaScript-Schleife mit Array-Zugriffen und Vergleichen

Zusammengefasst beschreibt das **Problem**, **was** erreicht werden soll. Der **Algorithmus** beschreibt allgemein, **wie** dies erreicht werden kann. Das **Programm** übersetzt diesen Lösungsweg schließlich in eine konkrete, ausführbare Sprache.

Übung: Algorithmus oder Programm?

Formuliere einen sprachunabhängigen Algorithmus zum Finden des größten Listeneintrags. Prüfe ihn auf Allgemeinheit, Eindeutigkeit, Ausführbarkeit, Endlichkeit, Terminierung und Korrektheit. Markiere danach, welche zusätzlichen Entscheidungen erst bei einer Umsetzung als Programm nötig werden.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

9 Rechnen mit integers – ein erster Algorithmus

In diesem Kapitel geht es nicht um einen Beweis, sondern um einen **Algorithmus**.
Wir wollen zeigen, wie man mit **integers** im Notizblock-Modell rechnen kann.
Das ist ein kleiner erster Schritt, um zu sehen: Schulmathematik kann durch die Funktionen einer Turing-Maschine (bzw. unserer Notizblock-Maschine) beschrieben werden – also durch **Lesen**, **Schreiben**, **Vergleichen** und **Springen**.

9.1 Grundidee

Wir legen die Ziffern **0 bis 9** im Speicher nebeneinander als Bereich an.
Dadurch kann der Prozessor innerhalb dieses Bereichs zählen, indem er von einer Zelle zur nächsten weitergeht.

Bei einer Aufgabe wie `2 + 5` könnte man den Speicherbereich für die Ziffern als Zählbereich verwenden: Die **3. Spalte** (= die Ziffer 2) wird ausgelesen, und anschließend werden über eine Schleife **5 weitere Spalten** nacheinander gelesen – also ein wiederholtes Zählen von einer Zelle zur nächsten.
Für den Anfang betrachten wir nur **ganze Zahlen** ohne Komma.

```
Zeile 1 (Ziffernbereich):
Spalte   [1,1] [1,2] [1,3] [1,4] [1,5] [1,6] [1,7] [1,8] [1,9] [1,10]
Ziffer      0     1    (2)    3     4     5     6    [7]    8      9
                       Start   .     .     .     .     Ziel
                               1     2     3     4      5   Schleifenschritte

Zeile 2 (Addition):
[2,1] = 2   [2,2] = '+'   [2,3] = 5   [2,4] = '='   [2,5] = 7
```

Die Schleife startet im Ziffernbereich bei der 2. Nach jedem der fünf Durchläufe geht der Prozessor genau eine Zelle weiter. Nach dem fünften Schritt liest er die 7 und schreibt sie als Ergebnis der Addition in Zeile 2.

9.2 Beispiel: Addition mit Übertrag

Wir addieren zwei integers, zum Beispiel `478` und `256`.
Die Zahlen (478 und 256) stehen zeichenweise im Notizblock, und der Prozessor arbeitet von rechts nach links.
Genauso wie auch die einzelnen Ziffern (0 bis 9) im Notizblock vermerkt sind, sodass wir wie bereits erwähnt (Beispiel: 2 + 5) wiederholt zählen können.
Jetzt müssen wir dies für jede Stelle der beiden Zahlen tun und dabei den Übertrag berücksichtigen.

```
A = 478
B = 256

Schritt 1: 8 + 6 = 14
          -> schreibe 4, merke Übertrag 1

Schritt 2: 7 + 5 + 1 = 13
          -> schreibe 3, merke Übertrag 1

Schritt 3: 4 + 2 + 1 = 7
          -> schreibe 7, kein weiterer Übertrag

Ergebnis: 734
```

Der Algorithmus benutzt also nur drei Dinge:

- **Lesen** der aktuellen Ziffern
- **Schreiben** der Ergebnisziffer
- **Merken** eines Übertrags im Hilfsblock

9.3 Beispiel: Subtraktion mit Übertrag / Ausleihen

Übung: Wie würde sich der Algorithmus anders beim Subtrahieren verhalten? Beachte dabei auch das wiederholte Zählen.

Hier sieht man bereits ein typisches Muster von Algorithmen:

- aktuelle Stelle ansehen
- Fall unterscheiden
- Hilfswert merken
- zur nächsten Stelle springen

9.4 Warum ist das ein Notizblock-Maschinen-Schritt?

Eine Notizblock-Maschine braucht keine mmagische Mathematik.
Sie arbeitet nur mit sehr einfachen Aktionen: lesen, schreiben, vergleichen und den nächsten Schritt auswählen.
Genau das machen wir hier auch.
Die Schulmathematik wird also nicht auf einmal anders – sie wird nur in kleine, ausführbare Schritte zerlegt.

9.5 Erkenntnis

Mit einem Notizblock und einem Prozessor kann man integers nicht nur speichern, sondern auch systematisch verarbeiten.
Das ist noch nicht die ganze Schulmathematik, aber ein wichtiger erster Baustein.
Später können wir darauf aufbauen, etwa mit Multiplikation, Division, Gleichungen und weiteren Verfahren.

10 Hilfsblock – Hilfsvariablen

Beim Programmieren braucht man oft **zusätzliche Variablen**, die nicht direkt zum Problem gehören, aber für die Berechnung notwendig sind.
Beispiele: *Zähler* in Schleifen, *Zwischenergebnisse* bei Berechnungen oder *Flags*, um sich zu merken, ob etwas gefunden wurde.

Um diese **Hilfsvariablen** klar von den eigentlichen **Daten** zu trennen, nutzen wir ein zweites kariertes Papier – das **Hilfsblatt**.
Zugriff erfolgt mit `h[Zeile, Spalte]` (h = Hilfsspeicher).

Typische Nutzfelder im Hilfsblatt:

- **Zähler und Zeiger** (z. B. Zeilen 1-5): *„Wo bin ich gerade?"*
  Beispiel: Schleifenzähler, Index in einer Liste
- **Sammelstellen für Werte** (z. B. Zeilen 6-10): *„Was setze ich gerade zusammen?"*
  Beispiel: Mehrere Ziffern zu einer Nummer oder einem Preis zusammensetzen
- **Zwischenwerte und Berechnungen** (z. B. Zeilen 6-10): *„Was berechne ich gerade?"*
  Beispiel: Temporäre Summe, Zwischenergebnis bei Division
- **Zustände und Merker** (z. B. Zeilen 11-15): *„Was habe ich gefunden/erreicht?"*
  Beispiel: Flag für "gefunden", aktuelles Minimum/Maximum

- **Werkzeuge und Konstanten** (z. B. Zeilen 16-25): *„Was brauche ich immer wieder?"*
  Beispiel: Ziffern 0-9 für Grundrechenarten, mathematische Konstanten

Beispiel 1: Schleifenzähler beim Durchgehen einer Liste

```
         | Sp.1 | Sp.2 | Sp.3 | Sp.4 | Sp.5 | Sp.6
---------+------+------+------+------+------+------
Zeile 1  | [1,1]| [1,2]| [1,3]| [1,4]| [1,5]| [1,6]   <- "Tobias" (6 Zeichen)
         | T    | o    | b    | i    | a    | s
Zeile 2  | [2,1]| [2,2]| [2,3]| [2,4]|      |          <- "Erik"   (4 Zeichen)
         | E    | r    | i    | k    |
Zeile 3  | [3,1]| [3,2]| [3,3]| [3,4]|      |          <- "Leon"   (4 Zeichen)
         | L    | e    | o    | n    |

        Jeder Buchstabe belegt eine eigene Zelle. Leere Zellen bleiben frei.
        Wir wollen jede Zeile dieser Liste durchgehen.
        Dazu müssen wir aber wissen, was die letzte Zeilennummer war, um mit der nächsten Zeile fortzufahren.
        Daher speichern wir uns die aktuelle Zeilennummer im Hilfsspeicher und können nachher ab dieser fortsetzen.

      Hilfsspeicher:

         | Spalte 1
---------+---------
Zeile 1  | 1 -> d.h. wir sind bei Tobias

         | Spalte 1
---------+---------
Zeile 1  | 2 -> d.h. wir sind bei Erik

         | Spalte 1
---------+---------
Zeile 1  | 3 -> d.h. wir sind bei Leon
```

Beispiel 2: Merken, ob etwas gefunden wurde

Wenn wir in der Bibliotheksliste die Nummer `97831` suchen, ist ein Merker hilfreich:

```
Hilfsspeicher:

       | Spalte 1
-------+-----------------
Zeile 11| gefunden = nein

Solange keine passende Nummer gefunden wurde:
  gefunden = nein
```

```
Sobald die zusammengesetzte Nummer 97831 ist:
  gefunden = ja
```

So kann sich der Prozessor merken, **ob die Suche erfolgreich war**, auch wenn er danach noch weitere Schritte ausführen muss.

Diese Trennung macht Programme **übersichtlicher** und hilft Anfängern zu verstehen, welche Daten zum Problem gehören und welche nur Hilfsmittel (Meta-Daten als Mittel zum Zweck) sind.

Übung: Hilfsvariablen aus dem Ablauf ableiten

Entwirf einen eigenen Lösungsweg zum Finden des kleinsten Preises. Beschreibe insbesondere den kreativen Ansatz, mit dem du einen bisher gelesenen Preis mit dem bisherigen Minimum vergleichst und das Minimum aktualisierst. Formuliere diese Idee anschließend als wiederverwendbares Werkzeug für dein persönliches Werkzeug-Portfolio.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

Übung: Weitere Informationen im Hilfsblock

Trage zusammen, welche Informationen außer Zählern und dem Merker *gefunden* noch im Hilfsblock abgelegt werden können. Ordne deine Vorschläge den Fragen „Wo bin ich?“, „Was weiß ich bereits?“, „Was berechne ich gerade?“ und „Bin ich fertig?“ zu und gib jeweils ein konkretes Beispiel für eine zellenweise Speicherung an.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

11 Programmatische Problemlösung mithilfe von Werkzeugen

Die folgenden vier Werkzeuge bilden den Kern unserer Notizblock-Denkweise.
Sie helfen beim systematischen Lösen von Problemen – von einfachen Einträgen bis zu vollständigen Algorithmen. Dabei verwenden wir immer dieselben Begriffe und Schritte: Problem und Probleminstanz, Ausgangszustand, gewünschter Zielzustand, Werkzeuge, Lösungsweg und erreichter Zielzustand.

Werkzeug	**Wirkung**	**Geeignete Situationen**
Schreiben	Werte werden gezielt in Zellen abgelegt oder überschrieben. Dadurch entstehen neue Zustände im Notizblock.	Wenn Daten neu eingetragen werden müssen (z. B. Startwerte), wenn Zwischenergebnisse festgehalten werden sollen oder wenn am Ende ein Ergebnis dokumentiert wird.
Lesen	Werte werden aus konkreten Zellen entnommen, ohne den Speicherinhalt zu verändern.	Wenn Entscheidungen vorbereitet werden, wenn mit vorhandenen Daten weitergerechnet wird oder wenn Eingaben geprüft und verarbeitet werden sollen.
Schleife	Ein Ablauf wird mehrfach wiederholt, bis alle relevanten Einträge bearbeitet sind oder eine Abbruchbedingung erreicht ist.	Bei Listen, Tabellen und Matrizen mit vielen ähnlichen Einträgen; bei wiederholtem Zählen; beim schrittweisen Suchen; generell dann, wenn derselbe Arbeitsschritt mehrfach nötig ist.
Bedingung	Der Ablauf verzweigt abhängig von einem Vergleich (z. B. gleich, kleiner, größer). So kann auf unterschiedliche Fälle passend reagiert werden.	Wenn zwischen Fällen unterschieden werden muss (Treffer/kein Treffer, gültig/ungültig, positiv/negativ), wenn Regeln nur unter bestimmten Voraussetzungen gelten oder wenn eine Schleife gezielt beendet werden soll.

Didaktischer Hinweis: In der Praxis werden diese Werkzeuge fast immer kombiniert, z. B. *lesen* → *bedingung prüfen* → *schreiben* innerhalb einer *Schleife*. Genau diese Kombination macht aus Einzelschritten einen vollständigen Algorithmus.

11.1 Demonstrationsbeispiel

11.1.1 Problem und Probleminstanz

Das allgemeine **Problem** lautet: Addiere eine Reihe von Zahlen und speichere ihre Summe.
Die konkrete **Probleminstanz** besteht aus den neun Zahlen 12, 7, 3, 5, 9, 4, 8, 6 und 2.

11.1.2 Ausgangszustand (Startsituation)

Im Notizblock stehen in den ersten 9 Zeilen zweistellige Zahlen, die addiert werden sollen. Jede Ziffer belegt eine eigene Zelle; einstellige Zahlen erhalten eine führende Null:
`[1,1]=1, [1,2]=2; [2,1]=0, [2,2]=7; ...; [9,1]=0, [9,2]=2`.
Dies entspricht den Zahlen 12, 7, 3, 5, 9, 4, 8, 6 und 2.

11.1.3 Gewünschter Zielzustand

Die Summe aller neun Zahlen steht **ab** `[10,1]`, wobei jede Ergebnisziffer eine eigene Zelle belegt. Für die Summe 56 gilt also: `[10,1]=5` und `[10,2]=6`. Daran erkennen wir, dass das Problem gelöst ist.

11.1.4 Werkzeuge und begründete Werkzeuganwendungen

- **Schleife:** Wir gehen mit einer Schleife über die Zeilen 1 bis 9.
  Warum? Weil wir genau diese 9 Zahlen systematisch addieren wollen.
- **Lesen:** In jedem Schleifendurchlauf lesen wir mit `integer(Zeile, 1, 2)` die beiden Ziffern der aktuellen Zahl.
  Warum? Nur so liegt die vollständige Zahl für die laufende Summe vor.
- **Addition:** Wir addieren die gelesene Zahl zur bisherigen Summe. Die Addition wurde zuvor aus den grundlegenden Werkzeugen Lesen, Schreiben, Vergleichen, Schleife und Hilfswerten aufgebaut und als wiederverwendbares Werkzeug in unser **Werkzeug-Portfolio** aufgenommen. Deshalb dürfen wir sie hier als bereits definiertes Werkzeug verwenden, ohne ihre inneren Schritte jedes Mal erneut auszuführen. Die laufende Summe bleibt dabei weiterhin ziffernweise im Hilfsblock gespeichert.
  Warum? Erst durch diese Werkzeuganwendung entsteht aus den gelesenen Einzelzahlen schrittweise die Gesamtsumme.
- **Schreiben:** Nach dem letzten Durchlauf schreiben wir das Gesamtergebnis ziffernweise ab `[10,1]`.
  Warum? Das Resultat wird dauerhaft im Notizblock gespeichert und ist für weitere Schritte verfügbar. Jede Ziffer belegt dabei genau eine Zelle.

Diese geordnete Folge der Werkzeuganwendungen bildet den **Lösungsweg**. Bei jeder Anwendung wird geprüft, ob sie im aktuellen Zustand ausführbar und für das Erreichen des Zielzustands sinnvoll ist.

11.1.5 Erreichter Zielzustand (Endsituation)

Nach der Verarbeitung steht im Notizblock:
`[10,1] = 5, [10,2] = 6`.
Das Ziel wurde erreicht: Aus Startwerten wurde durch geeignetes Kombinieren der Werkzeuge ein korrektes Ergebnis erzeugt und abgelegt.

12 Laufzeitanalyse: vom Schrittmodell zur Größenordnung

Zwei Lösungswege können denselben Zielzustand erreichen und trotzdem unterschiedlich aufwendig sein. Um sie fair zu vergleichen, müssen wir zuerst festlegen, **welche Kosten wir zählen**. Erst danach können wir die gezählten Schritte verallgemeinern.

12.1 Das Schrittmodell: Was kostet wie viel?

Ein **Kostenmodell** legt fest, welche Operationen als Schritte gelten und welches Gewicht sie erhalten. Ohne ein solches Modell ist die Aussage „dieser Algorithmus ist schnellerüngenau.

- **Vergleichsmodell:** Wir zählen nur Vergleiche. Dieses Modell eignet sich besonders für Suchverfahren.
- **Einfaches Schrittmodell:** Lesen, Schreiben, Vergleichen und Springen kosten jeweils einen Schritt.
- **Gewichtetes Schrittmodell:** Verschiedene Operationen dürfen unterschiedlich viel kosten, etwa Lesen = 2 und Vergleichen = 1.

Keines dieser Modelle ist grundsätzlich „das richtige". Ein Modell ist geeignet, wenn es für die untersuchte Frage die entscheidenden Kosten sichtbar macht. In den folgenden Suchbeispielen verwenden wir zunächst das **Vergleichsmodell**.

Wichtig

Erst das Kostenmodell festlegen, dann Schritte zählen. Ergebnisse aus unterschiedlichen Kostenmodellen dürfen nicht ungeprüft miteinander verglichen werden.

12.2 Konkrete Schrittzahl der linearen Suche

Probleminstanz: Suche Felix in `[Anna, Ben, Clara, David, Emma, Felix, Greta]` mit $n = 7$.

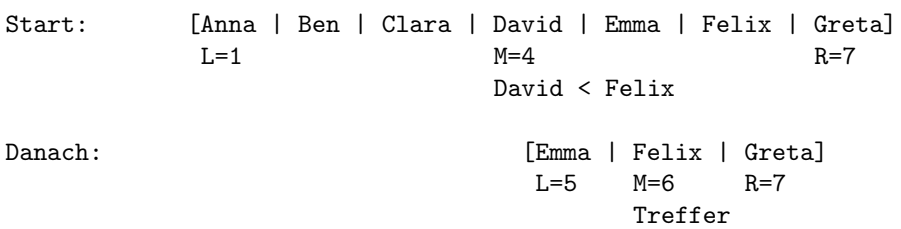
Die Einträge werden von links nach rechts mit Felix verglichen. Im Vergleichsmodell kostet jeder Vergleich genau einen Schritt. Für diese Instanz sind sechs Schritte nötig.

- **Best Case:** Felix steht am Anfang: $T_{best}(n) = 1$.
- **Worst Case:** Felix steht am Ende oder fehlt: $T_{worst}(n) = n$.
- **Average Case:** Bei gleich wahrscheinlichen Positionen sind ungefähr $T_{avg}(n) = (n + 1)/2$ Vergleiche nötig.

Best, Worst und Average Case sind keine verschiedenen Kostenmodelle. Sie beschreiben verschiedene Lagen der Eingabe **innerhalb desselben Modells**.

12.3 Konkrete Schrittzahl der Binärsuche

Die Binärsuche benötigt einen sortierten Notizblock. Sie prüft jeweils den mittleren Eintrag und verwirft danach eine Hälfte des noch möglichen Bereichs.



Die Grenzen `L` und `R` markieren den noch möglichen Bereich, `M` dessen Mitte. Nach dem Vergleich mit David kann die linke Hälfte einschließlich David ausgeschlossen werden. Der nächste betrachtete Bereich enthält nur noch Emma, Felix und Greta.

1. Setze die linke und rechte Grenze auf den gesamten Suchbereich.
2. Bestimme die Mitte und vergleiche den dortigen Eintrag mit der gesuchten Person.
3. Suche abhängig vom Ergebnis nur in der linken oder rechten Hälfte weiter.
4. Beende die Suche bei einem Treffer oder bei einem leeren Suchbereich.

Für Felix gilt: Zuerst wird David, danach Felix geprüft. Im Vergleichsmodell kostet diese Probleminstanz also zwei Schritte. Im Worst Case halbiert sich der Suchbereich so lange, bis nur noch ein Eintrag übrig ist:

$$n \to \frac{n}{2} \to \frac{n}{4} \to \frac{n}{8} \to \cdots \to 1$$

Bei $n = 1024$ sind höchstens ungefähr $\log_2(1024) = 10$ Vergleiche nötig.

12.4 Von der Schrittzahl zur asymptotischen Laufzeitordnung

Konkrete Schrittzahlen hängen vom gewählten Kostenmodell, von der Eingabe und teilweise von der Maschine ab. Für große Eingaben interessiert uns deshalb vor allem die Frage: **Wie wächst der Aufwand, wenn n wächst?**

Die asymptotische Laufzeitordnung beschreibt dieses langfristige Wachstumsverhalten. Dabei betrachten wir nicht mehr jeden einzelnen Schritt, sondern den dominierenden Anteil einer Kostenfunktion.

12.4.1 Die Idee der Asymptote: Kleingeld spielt keine Rolle

Angenommen, ein detailliertes Kostenmodell (konkrete Schrittzahl) liefert für eine lineare Suche

$$T(n) = 5n + 20.$$

Die zusätzlichen 20 Schritte und der Faktor 5 sind für eine konkrete Ausführung durchaus echte Kosten. Wenn n jedoch sehr groß wird, entscheidet vor allem der linear wachsende Anteil n über das Verhalten. Bildlich gesprochen: Beim langfristigen Wachstum ist **Kleingeld nicht ausschlaggebend**.

Deshalb fassen wir $5n + 20$, n und $100n + 3$ in derselben asymptotischen Ordnung $\mathcal{O}(n)$ zusammen. Entsprechend gehört eine binäre Suche zu $\mathcal{O}(\log n)$.

Kleingeld spielt keine Rolle

Die O-Notation behauptet nicht, dass Konstanten bei einer konkreten Messung wertlos sind. Sie blendet sie bewusst aus, wenn nur das Wachstum für sehr große Eingaben verglichen werden soll.

12.5 Wachstumsordnungen vergleichen

Anzahl n	Lineare Suche: n	Binärsuche: ungefähr $\log_2 n$
10	10 Vergleiche	4 Vergleiche
100	100 Vergleiche	7 Vergleiche
1.000	1.000 Vergleiche	10 Vergleiche
1.000.000	1.000.000 Vergleiche	20 Vergleiche

Die lineare Suche hat im Worst Case die Ordnung $\mathcal{O}(n)$, die Binärsuche $\mathcal{O}(\log n)$. Dieser Vergleich setzt allerdings voraus, dass die Daten für die Binärsuche bereits sortiert sind. Auch Voraussetzungen eines Algorithmus gehören daher zur Bewertung eines Lösungswegs.

Übung: Vom Zählen zum Wachstum

Gegeben sind die Kostenfunktionen $T_1(n) = 3n + 8$, $T_2(n) = n^2 + 2n$ und $T_3(n) = 4\log_2(n) + 20$. Berechne zunächst die konkreten Kosten für $n = 10$ und $n = 1000$. Streiche danach gedanklich das „Kleingeldünd ordne die Funktionen $\mathcal{O}(n)$, $\mathcal{O}(n^2)$ oder $\mathcal{O}(\log n)$ zu.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

12.6 Verbindung zur JavaScript-Umsetzung

Auch ein JavaScript-Programm kann nach verschiedenen Kostenmodellen untersucht werden. Wir können beispielsweise nur die Auswertung einer Suchbedingung zählen oder zusätzlich Array-Zugriffe, Zuweisungen und Schleifensprünge. Wichtig ist nicht, möglichst viele Dinge zu zählen, sondern das Modell vor der Analyse eindeutig anzugeben.

13 Sanfter Umstieg in eine Hochsprache (JavaScript)

Der Übergang erfolgt in kleinen Stufen. In jeder Stufe kommt möglichst nur **eine neue Idee** hinzu. Alles andere bleibt zunächst so, wie es aus dem Notizblock-Modell bekannt ist. Dadurch kann jede neue Schreibweise auf ihre bereits verstandene Bedeutung zurückgeführt werden.

13.1 Stufe 1: Gleicher Speicher, andere Schreibweise

Zum sanften Übergang bilden wir den karierten Notizblock zunächst **mithilfe von JavaScript** als Matrix ab. Zunächst bleiben sogar die einzelnen Zeichen erhalten. Nur die Schreibweise der Adresse ändert sich.
Wie diese Matrix in JavaScript initialisiert wird, ist an dieser Stelle nebensächlich und wird als vorbereitetes Werkzeug vorgegeben. Darüber muss zunächst nicht weiter nachgedacht werden; entscheidend sind die bekannten Zellen und ihre Adressen.

Notizblock	JavaScript
`[1,1] = 'M'`	`notizblock[0][0] = 'M';`
`[1,2] = 'a'`	`notizblock[0][1] = 'a';`

JavaScript beginnt beim Zählen mit 0. Daher entspricht Notizblock-Adresse `[Zeile, Spalte]` in JavaScript der Adresse `[Zeile - 1][Spalte - 1]`.

```
let notizblock = Array.from(
  { length: 10 },
  () => Array(10).fill(null)
);

notizblock[0][0] = 'M';
notizblock[0][1] = 'a';
notizblock[0][2] = 't';
```

Entscheidend ist zunächst allein die Übersetzung der Adressen.

Übung: Adressen übersetzen

Übertrage `[2,3] = 'k'`, `[4,1] = 7` und `[10,5] = ja` in JavaScript. Formuliere anschließend selbst zwei JavaScript-Zugriffe und übersetze sie zurück in die Notizblock-Schreibweise.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

13.2 Stufe 2: Lesen und Schreiben

Auch die Grundwerkzeuge bleiben gleich. Eine Zuweisung schreibt einen Wert, die Verwendung einer Adresse auf der rechten Seite liest ihn.

```
notizblock[0][0] = 4;                        // schreiben
notizblock[0][1] = 7;                        // schreiben
notizblock[0][2] = notizblock[0][0]
                   + notizblock[0][1];       // lesen und schreiben
```

Der Zielzustand lautet anschließend: In `notizblock[0][2]` steht der Wert 11. Neu ist nur die formale Schreibweise; Ausgangszustand, Werkzeuge und Zielzustand sind bereits bekannt.

Übung: Zustände sichtbar halten

Zeichne den Notizblock vor und nach jeder der drei Anweisungen. Markiere bei jedem Schritt, welche Zellen gelesen und welche geschrieben werden.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

13.3 Stufe 3: Mehrere Zeichen zu einem Wert zusammenfassen

Erst jetzt vereinfachen wir die Darstellung. Ein JavaScript-String darf ein ganzes Wort und eine Number eine ganze Zahl enthalten. Die Einzelzeichen verschwinden nicht; JavaScript verwaltet ihre interne Darstellung für uns.

```
// Zeichenweise, wie auf dem bisherigen Notizblock
notizblock[0][0] = 'M';
notizblock[0][1] = 'a';
notizblock[0][2] = 't';
notizblock[0][3] = 'h';
notizblock[0][4] = 'e';

// Neue Abstraktion: ein zusammengefasster Wert pro Zelle
notizblock[1][0] = 'Mathe 7';
notizblock[1][1] = 97831;
notizblock[1][2] = 1.99;
```

Diese Stufe verändert die **Datenrepräsentation**, aber noch nicht den Lösungsweg. Durch die Zusammenfassung benötigt die Bibliotheksliste nur noch die Spalten Titel, Nummer und Preis.

Übung: Abstraktion begründen

Speichere den Eintrag „Physik, Nummer 93410, Preis 22,49ßuerst zeichenweise und danach mit drei zusammengefassten Werten. Vergleiche den benötigten Platz und erkläre, welche Details die zweite Darstellung absichtlich verbirgt.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

13.4 Stufe 4: Wiederholen und Entscheiden

Die bekannte Schleife und Bedingung erhalten nun ihre JavaScript-Schreibweise. Der Zeilenzähler bleibt zunächst sichtbar im Hilfsblock.

```
let hilfsblock = Array.from(
  { length: 10 },
  () => Array(10).fill(null)
);

hilfsblock[0][0] = 97831; // gesuchte Nummer
hilfsblock[0][1] = false; // gefunden
```

```
hilfsblock[0][2] = 0;     // aktuelle Zeile

while (hilfsblock[0][2] <= 1) {
  if (notizblock[hilfsblock[0][2]][1] === hilfsblock[0][0]) {
    notizblock[9][0] = notizblock[hilfsblock[0][2]][0];
    notizblock[9][1] = notizblock[hilfsblock[0][2]][1];
    notizblock[9][2] = notizblock[hilfsblock[0][2]][2];
    hilfsblock[0][1] = true;
  }

  hilfsblock[0][2] = hilfsblock[0][2] + 1;
}
```

Die Klammern legen eindeutig fest, welche Anweisungen wiederholt und welche nur unter einer Bedingung ausgeführt werden. Inhaltlich bleibt der Ablauf derselbe: Zeile lesen, Nummer vergleichen, gegebenenfalls Ergebnis und Merker schreiben, zur nächsten Zeile gehen.

Übung: Ablaufprotokoll statt Raten

Führe das Programm für die gesuchte Nummer 93410 von Hand aus. Notiere nach jedem Schleifendurchlauf aktuelle Zeile, gelesene Nummer und den Merker *gefunden*. Wiederhole die Übung mit einer Nummer, die nicht vorhanden ist.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

13.5 Stufe 5: Das Tätigkeitenblatt wird zur Funktion

Ein wiederverwendbarer Ablauf des Tätigkeitenblatts entspricht in JavaScript einer Funktion. Erst nachdem der Ablauf verstanden und getestet wurde, erhält er einen Namen und kann als Baustein verwendet werden.

```
function sucheNummer(notizblock, gesuchteNummer) {
  let aktuelleZeile = 0;

  while (aktuelleZeile < 2) {
    if (notizblock[aktuelleZeile][1] === gesuchteNummer) {
      return aktuelleZeile;
    }
    aktuelleZeile = aktuelleZeile + 1;
  }

  return -1;
}

let gefundeneZeile = sucheNummer(notizblock, 97831);
```

Hier wird der Hilfsblock teilweise durch lokale Variablen ersetzt. Das ist keine neue Art des Denkens, sondern eine kompaktere Darstellung derselben Hilfswerte. Der Rückgabewert `-1` beschreibt den Zielzustand „kein Eintrag gefunden".

Übung: Vom Tätigkeitenblatt zur Funktion

Beschreibe zuerst ein Tätigkeitenblatt *preis_lesen(zeile)* mit Parametern, Werkzeuganwendungen und Ergebnis. Übersetze es anschließend in eine JavaScript-Funktion. Prüfe die Funktion mit einer gültigen und einer ungültigen Zeile.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

Roter Faden

JavaScript führt keine neue Form der Problemlösung ein. Notizblock, Hilfsblock, Werkzeuge und Tätigkeitenblatt werden schrittweise zu Matrix, Variablen, Anweisungen und Funktionen.

14 Zusammenhang Informatik - Mathematik

Die Schulmathematik – Arithmetik, Algebra, Analysis, Stochastik, Statistik – scheint nur bedingt was mit Informatik zu tun zu haben. Dennoch lassen sich alle diese Probleme prinzipiell im Notizblock-Modell lösen.

Die Idee dahinter:

- Das Notizblock-Modell hat einen Speicher (den Notizblock) und elementare Befehle wie Lesen, Schreiben, Vergleichen und Springen.
- Komplexe Rechenoperationen wie Addition, Subtraktion, Multiplikation oder Division lassen sich Schritt für Schritt aus diesen einfachen Befehlen aufbauen.
- Variablen der Mathematik entsprechen dabei Speicheradressen im Notizblock, Bedingungen und Schleifen werden über Vergleichs- und Sprungbefehle umgesetzt.

Mächtigkeit: Das Notizblock-Modell ist als vereinfachtes Rechenmodell Turing-vollständig interpretierbar – das bedeutet, dass alles, was algorithmisch lösbar ist, damit dargestellt werden kann. Damit kann die gesamte Schulmathematik schrittweise ausgeführt werden.

14.1 Gleichungen über Zahlen

Ein typisches Beispiel aus der Schulmathematik ist das Lösen von Gleichungen:

- Algebraisch: x + 3 = 7 → x = 7 - 3
- Imperativ / (klassisch) programmatisch: Schrittweise Werte testen oder hochzählen, bis die Gleichung erfüllt ist.

Beide Paradigmen können auf unser Notizblock-Modell abgebildet werden – entweder durch Beschreibung und Anwendung von Umkehrfunktionen oder durch schrittweise Verarbeitung mit Funktionsaufrufen und Vergleichen.

14.1.1 Warum Umkehrfunktionen häufig sinnvoller sind

Bei der algebraischen Variante wird die ursprüngliche Operation gezielt rückgängig gemacht. Aus

$$x + 3 = 7$$

wird durch Anwendung der Umkehrfunktion der Addition unmittelbar

$$x = 7 - 3 = 4.$$

Wenn Addition und Subtraktion bereits als Werkzeuge im Werkzeug-Portfolio liegen und im einfachen Kostenmodell jeweils als eine Werkzeuganwendung gezählt werden, benötigt dieser Lösungsweg unabhängig vom Wert auf der rechten Seite nur eine feste Anzahl von Schritten. Seine Laufzeit ist in diesem Modell $\mathcal{O}(1)$.

Die imperative Variante könnte dagegen bei einem Startwert beginnen und mögliche Werte nacheinander testen:

```
x = 0: 0 + 3 = 7? nein
x = 1: 1 + 3 = 7? nein
x = 2: 2 + 3 = 7? nein
x = 3: 3 + 3 = 7? nein
x = 4: 4 + 3 = 7? ja
```

Jeder weitere Kandidat benötigt mindestens eine Berechnung, einen Vergleich und gegebenenfalls einen Schleifenschritt. Liegt die Lösung nach n zu prüfenden Kandidaten, wächst der Aufwand linear mit n; die Suche besitzt dann die Laufzeitordnung $\mathcal{O}(n)$.

Struktur statt Ausprobieren

Eine bekannte Umkehrfunktion nutzt die Struktur der Gleichung und führt gezielt zur Lösung. Eine imperative Suche probiert mögliche Lösungen aus. Deshalb ist die algebraische Variante meist erheblich effizienter, besonders wenn der mögliche Wertebereich groß ist.

Dieser Vergleich gilt für das hier gewählte Kostenmodell. Bei sehr großen Zahlen haben auch arithmetische Werkzeuge innere Kosten. Außerdem muss eine passende Umkehrfunktion bekannt und unter den gegebenen Bedingungen anwendbar sein. Fehlt sie oder lässt sich die Gleichung nicht direkt umstellen, kann ein iteratives oder näherungsweises Verfahren dennoch sinnvoll sein.

Übung: Eine Gleichungsumstellung programmatisch ausführen

Entwickle ein JavaScript-Programm, das einfache Gleichungen mit einer Unbekannten x auf zwei Arten löst. Verwende zunächst Gleichungen der Formen $x + a = b$, $x - a = b$, $x \cdot a = b$ und $x/a = b$.

1. **Algebraisch mit automatischer Umstellung:** Das Programm erhält die Gleichung beziehungsweise ihre Bestandteile als Eingabe. Es erkennt selbstständig die verwendete Operation, wählt aus einem Werkzeug-Portfolio die passende Umkehrfunktion aus und stellt die Gleichung nach x um. Aus $x + 3 = 7$ soll es beispielsweise zuerst $x = 7 - 3$ und anschließend $x = 4$ erzeugen. Die Umstellung `x = b - a` darf also nicht bereits fest im allgemeinen Lösungsablauf vorausgesetzt werden. Beschreibe Ausgangszustand, Auswahl des Werkzeugs, Umformungsschritt und Zielzustand.
2. **Imperativ:** Beginne mit einem Startwert für x, teste in einer Schleife `x + a === b` und verändere x schrittweise, bis die Gleichung erfüllt ist. Beschreibe auch hier Ausgangszustand, verwendete Werkzeuge und Zielzustand.

Beide JavaScript-Varianten sollen für verschiedene Werte von a und b funktionieren. Erweitere auch die imperative Variante so, dass sie die jeweils eingegebene Operation prüfen kann. Zähle anschließend in einem selbst festgelegten Kostenmodell die benötigten Schritte und vergleiche die Laufzeit beider Lösungswege.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

14.2 Gleichungen über nicht-numerische Objekte: Farben

Gleichungen müssen nicht nur mit Zahlen gelöst werden. Wir können auch andere Objekte betrachten, z. B. Farben.

- Funktion **f**: vertauscht Rot und Cyan, Gelb und Blau, Grün und Magenta etc.
- Umkehrfunktion **f-1**: kehrt die Vertauschung wieder um.

Beispiel:

```
f(f(x)) = gelb | f-1 auf beide Seiten
f(x)     = f-1(gelb) = blau | f-1 erneut
x        = f-1(blau) = gelb
```

Ergebnis: `x = gelb`

Dieses Beispiel zeigt, dass Gleichungen auch über nicht-numerische Objekte gelöst werden können, solange die Funktion bijektiv ist.

14.3 Gleichungssysteme über bewegende Objekte

Betrachten wir zwei Autos auf einer Straße, die unterschiedliche Startpositionen und Geschwindigkeiten haben. Wir möchten bestimmen, wann sie sich treffen.

1. Mathematisch-algebraisch

- Auto A: WegA(t) = vA * t + sA,0
- Auto B: WegB(t) = vB * t + sB,0
- Gleichungssystem: WegA(t) = WegB(t)
- Lösung über Umkehrfunktionen: t = (sB,0 - sA,0) / (vA - vB)

2. Imperativ / programmatisch

- Wir starten bei t = 0 und erhöhen die Zeit schrittweise (hochzählen).
- In jedem Schritt wird WegA(t) und WegB(t) berechnet.
- Wenn WegA(t) = WegB(t), haben wir die Lösung gefunden.
- Dies entspricht dem klassischen Programmierparadigma, wie wir es z. B. in einer einfachen JavaScript-Umsetzung mit Schleifen formulieren könnten.

Dieses Beispiel illustriert:

- Gleichungssysteme können sowohl algebraisch über Umkehrfunktionen gelöst werden als auch imperativ durch Schritt-für-Schritt-Berechnung.
- Die imperativen Schritte lassen sich direkt im Notizblock-Modell ausdrücken, da jeder Schritt nur elementare Befehle benötigt (Werte lesen, schreiben, vergleichen, springen).
- So wird deutlich, dass das Notizblock-Modell oder eine einfache JavaScript-Umsetzung auch dynamische, zeitabhängige Probleme modellieren können.
- Damit können sowohl funktionale Lösungen über Umkehrfunktionen als auch imperative Lösungen über schrittweises Abarbeiten beschrieben werden.
- **Uns Menschen fällt imperatives (rezeptartiges) Denken sehr leicht.**

- Daher ist es oft sinnvoll, Probleme zunächst imperativ zu modellieren, bevor man versucht, eine abstrakte algebraische Lösung zu finden.
- **Selbstverständlich (und auch anhand unseres Beispiels mit den sich bewegenden Objekten) kann eine gedankliche Brücke (Framework) zwischen imperativem und algebraischem Denken gebaut werden.**

Für Lernende bedeutet dies: Das Notizblock-Modell und eine einfache JavaScript-Umsetzung mit Matrix sind universelle Werkzeuge, die weit über einfache Zahlenoperationen hinausgehen. Sie können auch Aufgaben lösen, die abstrakte Strukturen oder zeitabhängige Systeme betreffen, solange diese als Speicherinhalte modelliert werden.

14.4 Schlussfolgerungen

14.4.1 Was ist eine Schlussfolgerung?

Eine **Schlussfolgerung** ist ein logischer Ableitungsschritt, der neue Aussagen direkt aus bestehenden Prämissen ableitet. In komplexen Beweisen kann eine Schlussfolgerung aus mehreren aufeinander aufbauenden Einzelschritten bestehen. Jede Teilableitung ist dabei eine eigene Schlussfolgerung.

In vielen Beispielen werden Mengenlogik und Vergleiche angewandt, um Schlussfolgerungen zu ziehen. Eigentlich kann man aber mit jedem deterministischen System oder Modell Schlussfolgerungen betreiben.

14.4.2 Beispiele aus der natürlichen Sprache

Beispiel 1 – Tiere:

- Prämisse 1: Kein Tier ist müde.
- Prämisse 2: Einige Tiere sind nicht hungrig.
- Schlussfolgerung: Mindestens einige Tiere sind nicht müde.

Beispiel 2 – Sokrates:

- Prämisse 1: Alle Menschen sind sterblich.
- Prämisse 2: Sokrates ist ein Mensch.
- Schlussfolgerung: Sokrates ist sterblich.

Erkenntnis:

- Schlussfolgerungen sind die Bausteine logischen Denkens.
- Sie können auf Mengen, Eigenschaften oder konkrete Objekte angewendet werden.
- Auch ohne Zahlen lassen sich daraus neue Aussagen ableiten.

14.5 Beweisführung

14.5.1 Was ist eine Beweisführung?

Eine **Beweisführung** ist ein geordneter Prozess und damit ein Spezialfall der Problemlösung. Die allgemeine Struktur bleibt erhalten, erhält beim Beweisen aber fachlich genauere Namen:

- **Problem und Probleminstanz:** Allgemein soll die Gültigkeit einer Behauptung gezeigt werden; die konkrete Behauptung bildet die jeweilige Probleminstanz.
- **Ausgangszustand:** Bekannte Fakten, Annahmen oder Axiome heißen hier **Prämissen**.
- **Gewünschter Zielzustand:** Die **Behauptung oder These** ist nachvollziehbar aus den Prämissen hergeleitet.
- **Werkzeuge:** Schlussregeln und andere logische oder arithmetische Methoden, z. B. direkter Beweis, Beweis durch Widerspruch oder Induktion - TODO: näher auf diese eingehen.
- **Lösungsweg:** Die **Ableitung** ist eine geordnete Folge sinnvoller Werkzeuganwendungen, die von den Prämissen zur Behauptung führt.

Ein Beweis ist vollbracht, wenn die Behauptung aus den Prämissen zwingend folgt. Unter denselben Prämissen darf es keinen logisch zulässigen Weg geben, der zu einem Widerspruch zur Behauptung führt.

14.5.2 Beispiel 1 – Sokrates

Behauptung: Sokrates ist sterblich. *(Die Aufstellung der Behauptung startet die Beweisführung.)*

- Prämisse 1: Alle Menschen sind sterblich.
- Prämisse 2: Sokrates ist ein Mensch.
- Schlussfolgerung ist hier ein Werkzeug: Sokrates ist sterblich.

Erkenntnis: Der Großteil der Beweisführung besteht in der Anwendung von Schlussfolgerungen. Die Behauptung gibt den Rahmen vor.

Übung: Beweisführung als Problemlösung lesen

Ordne im Sokrates-Beispiel Problem, Probleminstanz, Ausgangszustand, Zielzustand, Werkzeug und Lösungsweg zu. Ersetze danach eine Prämisse so, dass die vorhandene Schlussfolgerung nicht mehr sinnvoll angewendet werden kann, und erkläre den entstandenen Bruch.

Lösungsweg skizzieren: Beschreibe, wie du vom Ausgangszustand zum Zielzustand gelangt bist. Nenne die eingesetzten Werkzeuge und begründe, warum du dich für sie entschieden hast.

14.5.3 Beispiel 2 – Arithmetik: Quadratische Faktorisierung

Behauptung: Für jede Zahl a gilt $\mathbf{a^2 - 1 = (a-1)(a+1)}$.

einige arithmetische Werkzeuge

- Ausklammern / Distributivgesetz: a(b+c) = ab + ac
- Faktorisierung von Summen/Differenzen: $a^2 - b^2 = (a-b)(a+b)$
- Potenzgesetze: $a^m \cdot a^n = a^{m+n}$
- Umformung von Termen: z. B. a2 - 12 → a2 - 1
- Substitution: Setze bekannte Werte ein (z. B. b=1)
- Induktionswerkzeuge: Für wiederkehrende Strukturen (z. B. Summenformeln)
- Übersetzen in arithmetische Sprache

- Polynom-Kongruenz
- aus einer unendlich zu testenden Menge eine endliche herbeiführen (Resteklassen, Transitivität)
- äquivalente Umformungen: wenn A = B gezeigt werden soll, dann A schrittweise so umstellen, dass B entsteht

Ausgewähltes Werkzeug Für diese Beweisführung wählen wir das Werkzeug **Faktorisierung von Differenzen: $a^2 - b^2 = (a-b)(a+b)$** - ein Spezialfall des Werkzeugs **Schablonenerkennung**.

Schlussfolgerung / Ableitung

1. Faktorisierung von Differenzen: $a^2 - b^2 = (a-b)(a+b)$
2. Setze $b = 1$ ein: $a^2 - 1^2 = (a-1)(a+1)$
3. Da $1^2 = 1$, folgt: $\mathbf{a^2 - 1 = (a-1)(a+1)}$.

14.5.4 Erkenntnis

- Beweisführung ist ein geordneter Rahmen, in dem Schlussfolgerungen als Werkzeuge eingesetzt werden.
- Schlussfolgerungen können einzeln oder in Ketten auftreten.
- Arithmetische Beweise nutzen Werkzeuge wie Faktorisierung, Potenzgesetze oder Summenregeln.
- Selbst scheinbar „magische“ Formeln werden nachvollziehbar, wenn Behauptung, ausgewähltes Werkzeug und Ableitung klar definiert sind.

Fazit: Wer Behauptung, Werkzeuge und Ableitung kennt, kann jede Beweisführung Schritt für Schritt aufbauen und verstehen.

14.5.5 Beispiel 3 - Teilbarkeit prüfen mit Polynom-Kongruenz

Behauptung (natürliche Sprache): **$2n^2 - 3n + 6$ ist durch 4 teilbar**.
Stimmt diese Behauptung für alle natürlichen Zahlen n?

Werkzeuganwendung: Übersetzen in arithmetische Sprache "Durch 4 teilbar"übersetzen wir in arithmetische Sprache:
$2n^2 - 3n + 6 \equiv 0 \pmod 4$

Werkzeuganwendung: Polynom-Kongruenz / Restklassen (Strukturerhalt des Terms)

Vorweg: Was bedeutet Kongruenz für mod 4? $n \equiv r \pmod 4$ genau dann, wenn $n = r + 4 \cdot m$ für ein ganzzahliges m.

Strukturgleichheit bei Modulo-Rechnungen

- `5 ≡ 9 (mod 4)` → beide Zahlen Rest 1 → gleiche Restklasse.
- `t(n) = n, t'(n) = n`

- – `t(5) = 5, t'(9) = 9`
  – `5` $\equiv$ `9 (mod 4)` $\rightarrow$ beide Restklasse 1, Struktur bleibt parallel.
- `t(n) = 2n, t'(n) = 2n`
- – `t(5) = 10, t'(9) = 18`
  – `5` $\equiv$ `9 (mod 4)`
  – `10` $\equiv$ `18 (mod 4)` $\rightarrow$ beide Restklasse 2, Struktur bleibt parallel.

Wie bei diesen Modulo-Rechnungen erkennbar, kann man nur anhand von n (ohne die Terme zu kennen) die Restklasse für mod 4 bestimmen.
Anders verhält es sich bei diesem Beispiel:

- `t(n) = 2n, t'(n) = 3n`
- – `t(5) = 10, t'(9) = 27`
  – `obwohl 5` $\equiv$ `9 (mod 4), ist 10 nicht` $\equiv$ `27 (mod 4)`

Hier unterscheidet sich die Restklasse zwischen den bloßen n-Werten und der Ausführung der Terme.
Die Struktur bleibt hier also nicht erhalten.
Merksatz: Wenn die Struktur gleich bleibt, genügt es, die Variable durch ihren Vertreter der Restklasse zu ersetzen anstelle des gesamten Terms.
In unserem Fall geht es darum zu wissen, wann der Term durch 4 teilbar ist.
Folgendes funktioniert für $2n^2 - 3n + 6 \equiv 0 \pmod 4$ leider nicht, weil dadurch der Term nicht mehr bekannt ist:

- $0 \bmod 4 = 0$
- $1 \bmod 4 = 1$
- $2 \bmod 4 = 2$
- ...

ABER: Wir wissen ja, dass wir n zum Testen verwenden können ohne dass die Struktur verloren geht.

Beweisführung: Prüfung aller Restklassen Wir prüfen jede Restklasse $n \equiv r \pmod 4$:

- $n \equiv 0 \pmod 4 \rightarrow 2 \cdot 0^2 - 3 \cdot 0 + 6 = 6 \rightarrow 6\%4 = 2 \rightarrow$ nicht teilbar
- $n \equiv 1 \pmod 4 \rightarrow 2 \cdot 1^2 - 3 \cdot 1 + 6 = 5 \rightarrow 5\%4 = 1 \rightarrow$ nicht teilbar
- $n \equiv 2 \pmod 4 \rightarrow 2 \cdot 2^2 - 3 \cdot 2 + 6 = 8 \rightarrow 8\%4 = 0 \rightarrow$ teilbar
- $n \equiv 3 \pmod 4 \rightarrow 2 \cdot 3^2 - 3 \cdot 3 + 6 = 15 \rightarrow 15\%4 = 3 \rightarrow$ nicht teilbar

Frage: Warum brauchen wir nur die Restklassen testen und nicht alle Zahlen?
Zur Erinnerung: Modulo gibt den Rest einer Division aus. (in unserem Beispiel zwischen einschließlich 0 und 3)
Auch wenn wir ein n $>$ 3 testen, bekommen wir auch nur ein Modulo-Ergebnis (einen Rest) zwischen einschließlich 0 und 3.

Nur im Falle der Resteklasse 0 wissen wir, dass es sich um eine Zahl handelt, die durch 4 teilbar ist.

Fazit: Nur für $n \equiv 2 \pmod 4$ (also bspw. für $n = 2, 6, 10, \dots$) ist der Term durch 4 teilbar.

Mit den Werkzeugen **Übersetzen in arithmetische Sprache** und **Polynom-Kongruenz** konnten wir die Teilbarkeit vollständig nachvollziehen und ohne aufwendige algebraische Tricks prüfen.

14.5.6 Beispiel 4 - Prüfung der Geradzahligkeit

Ausgangsbehauptung: **Wenn $2n^2 - 3n + 6$ durch 4 teilbar ist (siehe Beispiel 3), dann ist n gerade**.
Übung von: Lehrerfortbildung Baden-Württemberg – Übungen zum Beweisen

Aus der Restklassen-Analyse in Beispiel 3 wissen wir: Nur $n \equiv 2 \pmod 4$ liefert Teilbarkeit durch 4.

Geradzahligkeit prüfen mit Modulo 2 Eine Zahl ist gerade, genau dann, wenn $n \bmod 2 = 0$.

Wir setzen die Restklasse für die Teilbarkeit ein. Wichtig: Wir müssen **nicht alle Zahlen** prüfen, sondern nur die Vertreter der Restklassen. (siehe Polynom-Kongruenz), nämlich $n = 2$ für $n \equiv 2 \pmod 4$.

$(2 \cdot 2^2 - 3 \cdot 2 + 6) \bmod 2 = (8 - 6 + 6) \bmod 2 = 8 \bmod 2 = 0$

Fazit: Das Ergebnis ist $0 \rightarrow$ die Zahl ist gerade. Damit gilt die Behauptung.

Mit **Modulo 4** haben wir die Teilbarkeit ermittelt, mit **Modulo 2** die Geradzahligkeit überprüft. Dank der Restklassenanalyse ist das Verfahren übersichtlich und vollständig.

15 Wie kann man das Kochen lernen?

- Dieses Thema passt hier gut rein, weil dies ein gutes Beispiel ist, um meine Philosophie des Lernens über einen roten Faden zu verinnerlichen.
- Wirklich kochen kann man, wenn man keine Rezepte braucht, weil man sich ein gutes Rezept selber durch die physikalischen, biochemischen Grundlagen und durch Kenntnisse über die geschmackliche Komposition von Gewürzen und anderen Lebensmittel ableiten kann.
- Hat man diese Grundlagen einmal verstanden und häufig angewendet, entwickelt sich daraus ein mentales Framework. Dieses Framework enthält die wichtigsten Zusammenhänge und Erfahrungswerte in verdichteter Form. Dadurch muss man beim Kochen nicht jedes Mal alle physikalischen und biochemischen Prozesse bewusst herleiten, sondern kann schnell und zielgerichtet entscheiden.

16 Tipps zur Didaktik

16.1 Wo Übungen in diesem Lernweg sinnvoll sind

Übungen stehen vor allem an einem **Übergang zwischen zwei Darstellungen oder Denkstufen**. Dort zeigen sie, ob das Fundament verstanden wurde und auf eine neue Situation übertragen werden kann.

- **Nach einer neuen Darstellung:** Lernende stellen dieselbe Information selbst dar, lesen sie zurück und begründen ihre Struktur.
- **Nach einem neuen Werkzeug:** Lernende führen es aus und erklären, warum seine Anwendung im aktuellen Zustand sinnvoll ist.
- **Nach einem vollständigen Beispiel:** Lernende lösen eine ähnliche Probleminstanz auf einem eigenen, möglicherweise unkonventionellen Weg.

- **Vor einer stärkeren Abstraktion:** Lernende führen den bisher sichtbaren Ablauf von Hand aus. Erst danach wird er durch Datentypen, Funktionen oder eine Hochsprache verdichtet.
- **Nach einer Abstraktion:** Lernende übersetzen in beide Richtungen, etwa vom Notizblock zu JavaScript und wieder zurück.

Geeignete Übungen fragen daher nicht nur nach einem Ergebnis. Sie verlangen einen sichtbaren Ausgangszustand, einen Zielzustand, begründete Werkzeuganwendungen und eine Prüfung des erreichten Zustands. So bleibt Raum für eigene Lösungswege, ohne dass die fachliche Nachvollziehbarkeit verloren geht.

- Bei Übungen ist es wichtig, dass der Lernende seinen eigenen Weg nutzt, um zum Ziel zu kommen. Der Lehrende muss erkennen, dass trotzdem der Weg unkonventionell und ggf. auch umständlich ist, dieser auch zum Ziel führen könnte. Beispiel: Wenn der Schüler für den Anfang folgendes definiert: `var produktUndPreis = 'Brot für 1,99€'` und er das Produkt und den Preis ermitteln möchte, dann kann er das trotzdem herausfinden, auch wenn Produkt und Preis nicht in getrennte Variablen ausgewiesen ist.
- Die Erklärungen müssen häppchenweise (modular) stattfinden, sodass jeder das Thema nachvollziehen kann.
- Da Problemlösung auch nur eine Routine und keine Magie ist, kann jeder die zu lernenden Themen nicht nur logisch verstehen, sondern auch kreativ und problemlösend in diesem Bereich agieren. (**dazu wichtige Anmerkung:** Unser Gehirn kann die gleichen grundlegenden Operationen - Daten schreiben/lesen, Schleifen, Abfragen - wie ein Prozessor ausführen. Jeder noch so komplizierte Algorithmus könnte alleine mit diesen Grundoperationen umgesetzt werden.)
- Wenn man ein Konzept erlernt, müssen erst die dazugehörigen Fundamente verstanden werden. (viele WARUM-Fragen müssen gestellt werden)
  - Das bedeutet, es muss vom Konzept aus ein roter Faden bis hin zum Fundament verfolgt werden.
  - Beim Lösen von Aufgaben aus dem Konzept sollte man im Laufe der Zeit trotzdem ein gedankliches Framework entwickeln.
  - Das bedeutet, dass man beim Bearbeiten der Aufgabe nicht vom Fundament aus bis zum Konzept über die Zusammenhänge nachdenkt, sondern das gedankliche Framework beinhaltet die Struktur und die wichtigsten Tipps des Fundaments.
  - Damit kann man effizienter und zielgerichteter arbeiten, da das Framework als Leitfaden dient.
- Gut wäre es, ein Thema (bspw. Stochastik) über ein gemeinsames Fundament (bspw. Kombinatorik in Baumstruktur) zu vermitteln. Auch u.a. das mehrfache Falten von Papier kann auf Kombinatorik reduziert werden.